

REBOOT_VERIFICATION

Reboot-survival verification — procedure and honest status

Revision: 1 **Last modified:** 2026-09-01T18:10:00Z **Scope:** whether the boba stack comes back automatically after a host reboot, via `systemctl --user` with `linger` enabled.

Why this document exists

Reboot survival is the one requirement in this repair that **cannot be proven by an agent**. Host power-state transitions — reboot, poweroff, suspend, hibernate — are hard-banned (CONST-033) because this host runs mission-critical parallel agent and container workloads, and historical data loss has been traced to exactly those transitions.

So this document splits the requirement into what **has** been proven and the one step that remains operator-attended, with the exact commands.

Proven WITHOUT a reboot

Precondition	Status	Evidence
Units exist in-repo	PROVEN	<code>scripts/systemd/user/{boba.target,boba-stack.service,boba-webui-bridge.service,boba-resource-pressure-check.service,.timer}</code>
	PROVEN	

Precondition	Status	Evidence
Units no longer hardcode a nonexistent path		Previously all 14 path references pointed at /run/ media/milosvasic/DATA4TB/ Projects/boba, which does not exist. Now templated @@BOBA_REPO_ROOT@@, substituted at install time by scripts/boba-svc.sh. Guard: tests/pre_build/ test_systemd_unit_paths.sh (RED 14 failures → GREEN 0/38)
Units installed into ~/.config/ systemd/user/	PROVEN	systemctl --user list- unit-files \ grep boba returns all five
boba.target enabled	PROVEN	enabled
Timer enabled	PROVEN	Previously orphaned — enable only enabled the target, so WantedBy=timers.target was never realised and the hourly probe would never fire after reboot. Fixed
Linger enabled (user manager starts at boot without login)	PROVEN	loginctl show-user \$USER --property=Linger → Linger=yes; marker /var/ lib/systemd/linger/ milosvasic exists
Units are syntactically valid	PROVEN	systemd-analyze --user verify → rc=0 for all five
ExecStart referents exist and are executable	PROVEN	start.sh, stop.sh, qBitTorrent-go/bin/webui- bridge (ELF) all present + executable; --no-build is a real flag
The systemd start path actually brings the stack up	See §“Intermediate proof” below	

Necessary but NOT sufficient. Every item above can be true while the stack still fails to come back — because none of them exercises boot-time ordering.

Intermediate proof (safe, no reboot)

Starting a user unit is NOT a power-state transition and is safe to run. It exercises the *exact* ExecStart path systemd would use at boot, which is the closest achievable proof short of rebooting:

```
# Bring the stack down, then let SYSTEMD (not ./start.sh) bring it
up.
bash scripts/boba-svc.sh down
    # or: systemctl --user stop boba.target
systemctl --user start boba.target
systemctl --user status boba.target --no-pager
systemctl --user status boba-stack.service --no-pager
podman ps --format '{{.Names}}\t{{.Status}}'
```

Expected: boba-stack.service reaches active (exited) with RemainAfterExit=yes, and all four containers report healthy.

What this still does NOT prove: boot-time ordering. At real boot the user manager starts before/alongside networking and the podman socket, and the image pull must complete inside TimeoutStartSec=300. Those conditions exist only during an actual boot.

The remaining operator-attended step

Run at a moment of your choosing, when no critical work is in flight:

```
# 1. BEFORE rebooting – record the expected state
podman ps --format '{{.Names}}\t{{.Status}}' > /tmp/boba-pre-
reboot.txt
systemctl --user is-enabled boba.target
loginctl show-user "$USER" --property=Linger      # must be
Linger=yes

# 2. Reboot the host by your own normal means.
#     (An agent must never issue this – CONST-033.)

# 3. AFTER reboot, WITHOUT logging into a desktop session if you
    want to
#     prove linger specifically, and WITHOUT running ./start.sh:
systemctl --user status boba.target --no-pager
systemctl --user status boba-stack.service --no-pager
```

```
podman ps --format '{{.Names}}\t{{.Status}}'
```

4. Prove the user-visible capability, not just the containers:

```
curl -s -o /dev/null -w 'login=%{http_code}\n' \
  -H 'Referer: http://localhost:7186' \
  --data 'username=admin&password=admin' \
  http://localhost:7186/api/v2/auth/login # expect 204
```

```
curl -s -o /dev/null -w 'wrong=%{http_code}\n' \
  -H 'Referer: http://localhost:7186' \
  --data 'username=admin&password=WRONG' \
  http://localhost:7186/api/v2/auth/login # expect 401
  (auth enforced)
```

5. If anything failed, capture WHY before changing anything:

```
journalctl --user -u boba-stack.service -b --no-pager | tail -60
journalctl --user -u boba.target -b --no-pager | tail -20
```

Pass criteria

All four must hold: 1. boba-stack.service is active (exited) without manual intervention 2. All four containers healthy 3. admin/admin on :7186 returns **204** 4. A wrong password returns **401** — authentication is enforced, not bypassed

Known risks to watch for at first real boot

Risk	Symptom	Where to look
Podman socket not ready when the unit starts	start.sh fails early	journalctl --user -u boba-stack.service -b
Network not up → image pull fails	Unit times out at TimeoutStartSec=300	same
/home/milosvasic/Share/Misc (QBITTORRENT_DATA_DIR) not mounted yet	qbittorrent container fails its volume mount	podman logs qbittorrent
Config written by a still-running qBittorrent	Credentials appear reverted	qBittorrent rewrites qbittorrent.conf on shutdown — never edit it while the container runs

Honest boundary (§11.4.6)

Reboot survival is **NOT VERIFIED** as of this document. Every precondition an agent can establish has been established and proven; the end-to-end behaviour has not, and no amount of additional agent work can establish it. Anyone reading this should treat reboot survival as *designed and staged*, not *proven*, until step 3 above has been run and its output recorded here.